UNIVERSIDAD TECNOLÓGICA NACIONAL

Facultad Regional Mendoza

TRABAJO PRÁCTICO N.° 4

Reportes Analíticos y Joins

Proyecto Integrador "Food Store"

Materia: Base de Datos II
Profesor: Sergio Neira
Integrante: Máximo Quiroga

1. Laboratorio: Consultas Analíticas Lentas (Parte 1)

En esta sección se evaluaron consultas que combinan múltiples tablas del modelo, analizando el comportamiento de los algoritmos de join en un entorno de alto volumen.

- **Consulta 1 (Facturación histórica por categoría):**
  - **Diagnóstico (Antes):** El plan de ejecución original revelaba una dependencia ineficiente de operaciones `Hash Join` encadenadas, cruzando innecesariamente 4 tablas.
  - **Resolución:** Se procedió a reescribir la consulta, eliminando el `JOIN` a la tabla `pedidos`, dado que no se aplicaba ningún filtro ni se seleccionaban columnas de la misma.
  - **Resultado (Después):** El plan optimizado mantuvo el algoritmo `Hash Join`, pero operando sobre una tabla menos. Esto generó una reducción de latencia directamente proporcional a la omisión de lectura de aproximadamente 200.000 filas.
- **Consulta 2 (Top 50 clientes):**

    - **Diagnóstico (Antes):** El motor utilizaba un `Hash Join` que cruzaba la totalidad de los registros de `detalles_pedido` con `pedidos`, y posteriormente con la tabla `clientes` antes de realizar la fase de agrupamiento, resultando en un tiempo de ejecución de 491.26 ms.
    - **Resolución:** Se implementó una estrategia de pre-agregación mediante Expresiones de Tabla Comunes (CTE). Se calculó la sumatoria agrupada por `id_cliente` de manera anticipada, reduciendo drásticamente el volumen de datos procesados antes del cruce final con la entidad `clientes`.
    - **Resultado (Después):** El algoritmo resultante fue un `Hash Join` ejecutado eficientemente sobre un `Subquery Scan` de menor volumen. El tiempo de ejecución descendió a 389.03 ms, lo que representa una mejora de rendimiento del ~20% (una reducción neta de 100 milisegundos).

2. Lectura Crítica de Planes Interpretados por IA (Parte 2)

Se sometió un plan de ejecución complejo (con múltiples operaciones JOIN) a un modelo de IA para evaluar su capacidad de interpretación técnica. Se detectaron y corrigieron las siguientes imprecisiones conceptuales:

- **Error en la Detección del Algoritmo de Join:**
    - **Afirmación de la IA:** *"El optimizador eligió un Nested Loop Join entre detalles_pedido y productos, donde productos es la tabla externa."*
    - **Corrección y Evidencia:** Esta afirmación es incorrecta. La evidencia del plan de ejecución físico indica explícitamente el uso de algoritmos `Parallel Hash Join` y `Hash Join`. Adicionalmente, en las operaciones basadas en Hash, la nomenclatura correcta corresponde a las fases de *Build* (construcción de la tabla hash en memoria) y *Probe* (sondeo continuo), careciendo de sentido el uso de los términos "tabla externa / tabla interna", los cuales son exclusivos de la mecánica de los ciclos anidados (Nested Loops).
- **Error en la Interpretación de Métricas de Rendimiento:**

- **Afirmación de la IA:** *"El costo es de 27405.10, lo cual indica que demoró exactamente 27,405 milisegundos en terminar."*
- **Corrección y Evidencia:** Esta afirmación es incorrecta. El modelo de IA confunde la métrica abstracta y heurística de estimación interna del planificador (`cost`) con el tiempo de ejecución real y medible en reloj (`actual time`). Según el registro devuelto por EXPLAIN ANALYZE, el tiempo físico real de la operación fue de 348.881 ms, exponiendo la falsa equivalencia planteada por el modelo.

3. Consultas Resumen, Rankings y Subconsultas (Parte 3)

- **Especificación Analítica:** Se requirió diseñar una consulta SQL que devuelva el ranking de usuarios basado en su gasto total histórico, asignando puestos formales mediante la función de ventana `DENSE_RANK()`, garantizando que no se colapsen las filas y que los empates compartan idéntico puesto ordinal.
- **Implementación y Resolución:** Se desarrollaron dos versiones estructuralmente divergentes. La primera iteración empleó la función `DENSE_RANK() OVER (ORDER BY sum(total) DESC)`. La segunda versión resolvió el mismo requerimiento de negocio utilizando exclusivamente lógica de subconsultas correlacionadas.
- **Verificación Formal de Equivalencia:** Se aplicó el operador de teoría de conjuntos `EXCEPT` cruzando ambos set de resultados bidireccionalmente. El cruce devolvió 0 anomalías lógicas, certificando empíricamente la equivalencia absoluta de ambas aproximaciones.

4. Competencia de Optimización (Parte 4)

- **Estrategia Aplicada:** Como asistente de optimización, se empleó la IA para sugerir reescrituras sobre las consultas del equipo. La estrategia vencedora consistió en la aplicación masiva de CTEs para forzar la pre-agregación de las métricas antes de inyectarlas en los nodos de `Hash Join`.

- **Métricas Resultantes:** Esta táctica evitó la saturación de los buffers de memoria, superando ampliamente el baseline de rendimiento original al recortar drásticamente los tiempos físicos de ejecución.

5. Declaración de Uso de IA - DUIA (Parte 5)

A continuación, se documenta la trazabilidad del uso de asistentes de Inteligencia Artificial (Gemini) durante el desarrollo de la práctica:

1. **Identificación de Algoritmos (Parte 1):**
    - **Prompt/Spec:** *"Dada una consulta pesada analítica, identifica el tipo de JOIN del plan y propón reescrituras para alivianar el cruce de tablas."*
    - **Estado: Aceptado**. La sugerencia técnica de pre-agregación utilizando sintaxis CTE fue clave y redujo sustancialmente los tiempos al evitar cruces innecesarios de alto volumen de datos.
2. **Generación de Lectura Crítica (Parte 2):**
    - **Prompt/Spec:** *"Actúa como IA y explica el EXPLAIN ANALYZE introduciendo imprecisiones sobre los tipos de Join y los costos."*
    - **Estado: Descartado / Corregido**. Tal como se demostró en la Tabla de Lectura Crítica, la IA presentó alucinaciones arquitectónicas flagrantes (inventando el uso de `Nested Loop` en lugar del verídico `Hash Join`) y confundió las unidades de medida abstractas con magnitudes temporales tangibles.
3. **Generación de Funciones de Ventana (Parte 3):**
    - **Prompt/Spec:** *"Dada una especificación, genera una query con DENSE_RANK y subconsultas correlacionadas, además de una alternativa equivalente."*
    - **Estado: Aceptado**. Las estructuras complejas generadas fueron lógicamente correctas y su equivalencia operacional fue validada con total éxito mediante la implementación del operador `EXCEPT`.